# DOCUMENTO DE ARQUITECTURA DEL SOFTWARE
# DAS

**FitPaisa**

*Plataforma Web de Entrenamiento y Nutrición Personalizada*

## 1. Vista de Alto Nivel

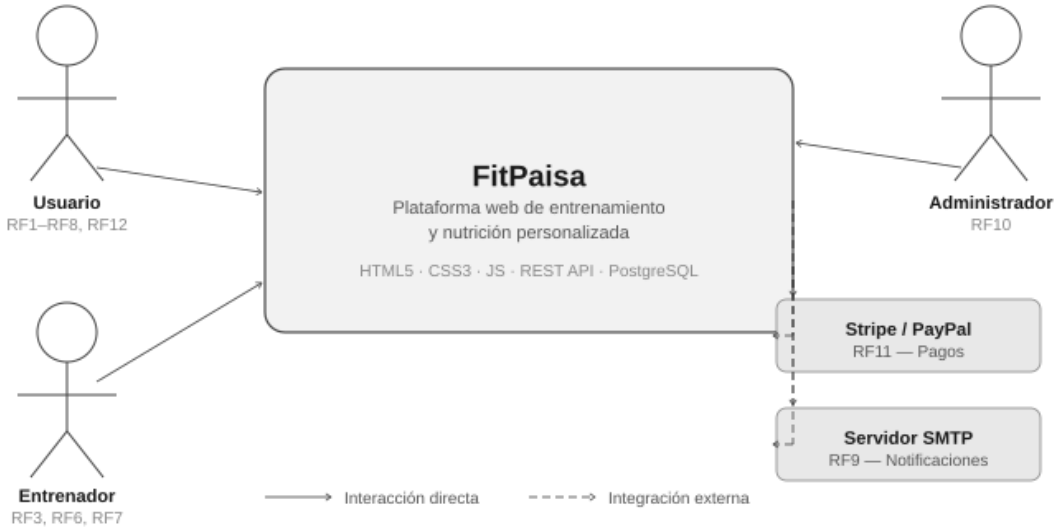
FitPaisa se concibe como una plataforma web integral para la gestión del entrenamiento físico y la nutrición personalizada. Visto como caja negra, el sistema recibe datos del usuario (perfil físico, comidas, sesiones de entrenamiento) y devuelve planes adaptados, estadísticas de progreso y comunicación directa con su entrenador.

El sistema opera de forma centralizada: el usuario interactúa con un entorno web responsive, los entrenadores supervisan el progreso de sus clientes en tiempo real y el administrador gestiona la plataforma desde un panel con permisos elevados. FitPaisa actúa como asistente activo y dinámico, integrando servicios externos para pagos y notificaciones.

### 1.1 Actores del sistema

| Actor | Rol | Capacidades principales |
|---|---|---|
| Usuario final | Cliente de la plataforma | Registrar comidas y entrenamientos · Consultar planes · Ver estadísticas · Suscripción Premium |
| Entrenador | Profesional supervisor | Crear y aprobar planes · Supervisar progreso · Mensajería con clientes |
| Administrador | Gestor de la plataforma | Gestionar usuarios, entrenadores y permisos · Supervisar el sistema |
| Sistema externo | Pasarelas de pago (Stripe/PayPal) | Procesamiento de suscripciones Premium (RF11) |
| Sistema externo | Servidor SMTP | Envío de notificaciones y recordatorios automáticos (RF9) |

### 1.2 Diagrama de vista de alto nivel




## 2. Patrones Arquitectónicos

Para FitPaisa se ha definido una arquitectura híbrida que combina el modelo Cliente-Servidor con el patrón de diseño Modelo-Vista-Controlador (MVC). Esta decisión se fundamenta en las características del proyecto: equipo unipersonal, entorno académico, requisito de respuesta inferior a 2 segundos y acceso exclusivamente web.

### 2.1 Evaluación de patrones candidatos

| Patrón | Ventajas para FitPaisa | Inconvenientes | Decisión |
|---|---|---|---|
| Cliente-Servidor | Separación clara front/back · Estándar web · Escalable | Requiere gestión de API | ✓ Adoptado |
| MVC | Separación datos/lógica/vista · Mantenible · Conocido | Mayor estructura inicial | ✓ Adoptado |
| Microservicios | Escalabilidad independiente · Equipos separados | Complejidad excesiva para equipo unipersonal | ✗ Descartado |
| Hexagonal | Alta mantenibilidad · Reglas de negocio aisladas | Sobreingeniería para el alcance académico del proyecto | ✗ Descartado |
| Event-Driven | Reactivo · Ideal para notificaciones en tiempo real | Infraestructura de mensajería compleja (Kafka, RabbitMQ) | ✗ Descartado |

### 2.2 Arquitectura adoptada: Cliente-Servidor + MVC

La arquitectura híbrida seleccionada organiza el sistema en dos grandes bloques y tres capas internas:

#### A. Estructura Cliente-Servidor

| Bloque | Componente | Responsabilidad |
|---|---|---|
| Cliente (Frontend) | Interfaz web responsive | Presentación de datos y captura de interacciones del usuario. Se ejecuta en el navegador (Chrome, Firefox, Edge). |
| Servidor (Backend) | API REST + Lógica + BD | Centraliza la lógica de negocio, el procesamiento de datos y la comunicación con servicios externos (Stripe, PayPal, SMTP). |

#### B. Implementación del patrón MVC

| Capa MVC | Componente en FitPaisa | Responsabilidad |
|---|---|---|
| Modelo | Lógica de negocio + Base de datos | Gestiona perfiles físicos, planes de entrenamiento, cálculo de macronutrientes e integridad de los datos almacenados. |
| Vista | Frontend (HTML5/CSS3/JS) | Renderiza el dashboard, gráficos de progreso, formularios de registro (máximo 3 pasos) y el chat de mensajería interna. |
| Controlador | Backend API | Actúa como intermediario: recibe peticiones de la Vista, aplica las reglas del Modelo y gestiona sesiones, roles y suscripciones. |

### 2.3 Justificación de la decisión

- Adecuación al equipo: la arquitectura MVC es manejable por un desarrollador unipersonal sin necesidad de orquestación de servicios.
- Cumplimiento de RF12: el patrón MVC facilita flujos de registro de máximo 3 pasos mediante controladores dedicados.
- Rendimiento: la separación de capas permite optimizar cada nivel de forma independiente para cumplir el requisito de carga en < 2 segundos.
- Escalabilidad futura: la separación Cliente-Servidor permite migrar a microservicios en el futuro sin reestructurar la lógica de negocio.

## 3. Decisiones Tecnológicas

Las decisiones tecnológicas de FitPaisa se han tomado priorizando la compatibilidad con el entorno académico, la madurez de las tecnologías y el cumplimiento de los requisitos no funcionales de la ERS.

### 3.1 Frontend

| Tecnología | Justificación |
|---|---|
| HTML5 | Estándar web universal. Soporte nativo en todos los navegadores objetivo (Chrome, Firefox, Edge). Estructura semántica de todas las vistas de la aplicación. |
| CSS3 | Diseño responsive sin dependencias externas. Garantiza usabilidad en cualquier tamaño de pantalla sin frameworks adicionales. |
| JavaScript | Lenguaje nativo del navegador. Gestiona la interactividad, la validación de formularios en cliente y las peticiones asíncronas a la API REST del backend. |
| JSP (JavaServer Pages) | Tecnología de plantillas Java EE para la generación dinámica de HTML en el servidor. Permite integrar lógica de presentación con los datos del backend Java directamente. |

*Criterio de selección: tecnologías conocidas por el desarrollador (HTML, CSS, JS, JSP) que no generan dependencias de frameworks externos y son compatibles con el entorno Java del backend.*

### 3.2 Backend

| Área | Tecnología | Justificación |
|---|---|---|
| Lenguaje principal | Java | Lenguaje dominado por el desarrollador. Tipado estático, robusto y con amplio ecosistema para aplicaciones web empresariales. |
| Framework web | Java EE / Servlets | Tecnología nativa de Java para la gestión de peticiones HTTP. Encaja directamente con JSP para el patrón MVC sin dependencias externas. |
| Scripting auxiliar | PHP | Utilizado puntualmente para tareas de scripting ligero o integración de componentes web específicos donde resulte más ágil que Java. |
| Scripts de utilidad | Python | Empleado para scripts de procesamiento de datos, generación de informes o tareas de mantenimiento fuera del flujo principal de la aplicación. |

| Área | Tecnología | Justificación |
|---|---|---|
| Autenticación | JWT (JSON Web Token) | Estándar abierto (RFC 7519) para sesiones sin estado. Implementable desde Java con librerías estándar (jjwt). Expiración configurable. |
| Hash de contraseñas | bcrypt | Función adaptativa resistente a ataques de fuerza bruta. Disponible en Java mediante librerías estándar. Factor de coste ≥ 12. |
| API | REST sobre HTTPS | Estilo arquitectónico desacoplado. Las respuestas JSON permiten que el Frontend JS/JSP consuma los datos sin acoplamiento tecnológico. |

### 3.3 Base de datos

| Tecnología | Cuándo se usa | Justificación |
|---|---|---|
| MySQL | Datos relacionales principales: usuarios, perfiles, planes, comidas, pagos, mensajes. | Sistema relacional conocido por el desarrollador. Transacciones ACID, integridad referencial y soporte nativo en entornos Java (JDBC/JPA). Ideal para datos estructurados con relaciones bien definidas. |
| MongoDB | Datos no estructurados o semiestructurados: logs de actividad, registros de estadísticas, histórico de notificaciones. | Base de datos documental flexible. Permite almacenar documentos JSON sin esquema fijo, adecuada para colecciones con estructura variable o de alto volumen de escritura. |

*Estrategia de persistencia híbrida: MySQL como base de datos principal para todos los datos transaccionales críticos (usuarios, planes, pagos), y MongoDB como almacén secundario para datos de naturaleza documental o con estructura variable.*

### 3.4 Servicios externos

| Servicio | Proveedor | Propósito | RF |
|---|---|---|---|
| Pasarela de pagos | Stripe y/o PayPal | Procesamiento seguro de suscripciones Premium. Cumple PCI-DSS. Los datos de tarjeta nunca pasan por los servidores propios. | RF11 |
| Envío de correos | SMTP / Email API | Envío de notificaciones automáticas, recordatorios de | RF9 |

| Servicio | Proveedor | Propósito | RF |
|---|---|---|---|
| | | actividades y emails de recuperación de contraseña. | |
| Certificado TLS | SSL/TLS 1.2+ | Cifrado de todas las comunicaciones entre cliente y servidor. Requerido por la ERS para protección de datos en tránsito. | RNF |

### 3.5 Infraestructura de despliegue

| Componente | Tecnología | Propósito |
|---|---|---|
| Servidor de aplicaciones | Apache Tomcat | Servidor Java EE para despliegue de Servlets y JSP. Entorno nativo para aplicaciones Java web. |
| Base de datos relacional | MySQL | Persistencia principal con integridad ACID. Soporte nativo en Java mediante JDBC/JPA. |
| Base de datos documental | MongoDB | Almacén secundario para datos no estructurados (logs, estadísticas, notificaciones). |
| Seguridad TLS | Certificado SSL/TLS | Cifrado obligatorio de todas las comunicaciones cliente-servidor (TLS 1.2 o superior). |

## 4. Restricciones Arquitectónicas

### 4.1 Restricciones técnicas

| Restricción | Descripción | Origen |
|---|---|---|
| Acceso exclusivamente web | El sistema debe ser accesible únicamente a través de navegador web. No se desarrollará aplicación nativa de escritorio ni móvil. | ERS — Restricciones de diseño |
| Compatibilidad de navegadores | Compatibilidad obligatoria con las versiones actuales de Chrome, Firefox y Edge. | ERS — Restricciones de diseño |
| Cifrado en tránsito | Toda transmisión de datos debe realizarse bajo protocolos TLS 1.2 o superior. Prohibido el uso de HTTP sin cifrar en producción. | ERS — Atributos del sistema |

| Restricción | Descripción | Origen |
|---|---|---|
| API REST | Toda comunicación Frontend-Backend se realizará mediante API REST sobre HTTPS con respuestas en formato JSON. | DAS — Patrón arquitectónico |
| Autenticación sin estado | La gestión de sesiones se implementará mediante tokens JWT. No se utilizarán sesiones de servidor (cookies de sesión). | DAS — Decisión tecnológica |
| Sin almacenamiento de pagos | Los datos de tarjeta de pago nunca se almacenarán ni procesarán en los servidores propios. Se delega íntegramente a Stripe/PayPal. | ERS — RF11, Seguridad |
| Tecnologías web estándar | El sistema se desarrollará con tecnologías web estándar, sin dependencias de frameworks propietarios, por restricción del entorno académico. | ERS — Restricciones de diseño |

### 4.2 Restricciones de negocio

| Restricción | Descripción | Métrica |
|---|---|---|
| Disponibilidad | El sistema debe mantener un 99,5% de tiempo de actividad mensual. | ≤ 3,6 horas de caída al mes |
| Rendimiento | El panel principal debe cargarse en menos de 2 segundos bajo condiciones normales de uso. | < 2 s en carga del dashboard |
| Usabilidad | El flujo de registro y primer acceso no puede superar los tres pasos (RF12). | ≤ 3 pasos hasta acceso completo |
| Privacidad | Cumplimiento de normativas básicas de protección de datos personales y sensibles de salud. | Datos de salud cifrados en BD |
| Alcance funcional | Las posibles mejoras futuras se limitarán a la ampliación de funcionalidades dentro del entorno web. No se contempla aplicación móvil nativa. | Sin app nativa |
| No diagnóstico médico | El sistema no sustituye asesoramiento médico ni realiza diagnósticos clínicos. | Aviso legal en la plataforma |

## 5. Interfaces Externas

FitPaisa se integra con los siguientes sistemas y servicios externos. Todas las comunicaciones se realizan sobre HTTPS con validación de firma en los eventos entrantes (webhooks).

### 5.1 Interfaz de usuario

| Aspecto | Especificación |
|---|---|
| Tipo de interfaz | Aplicación web responsive accesible desde navegador sin instalación. |
| Navegadores soportados | Chrome, Firefox y Edge en sus versiones actuales. |
| Criterios de diseño | Usabilidad con flujos de máximo 3 pasos (RF12). Tiempo de carga del panel < 2 segundos. |
| Seguridad cliente | Token JWT almacenado de forma segura en el cliente. Renovación automática antes de expiración. |

### 5.2 APIs de terceros

| Sistema externo | Tipo de integración | Protocolo | Descripción |
|---|---|---|---|
| Stripe | Pasarela de pagos | API REST HTTPS + Webhook | Generación de sesiones de checkout. Recepción de eventos de confirmación/rechazo de pago mediante webhook con validación de firma HMAC. |
| PayPal | Pasarela de pagos | API REST HTTPS + Webhook | Alternativa a Stripe para procesamiento de suscripciones. Mismo flujo de checkout y webhook. |
| SMTP/API | Servicio de correo | SMTP / REST Email API | Envío de notificaciones, recordatorios de actividades y emails transaccionales (confirmación de pago, bienvenida, recuperación de contraseña). |

### 5.3 Protocolos de comunicación

| Comunicación | Protocolo | Cifrado | Tipo |
|---|---|---|---|
| Frontend ↔ Backend | HTTP/REST | TLS 1.2+ | Síncrona |

| Comunicación | Protocolo | Cifrado | Tipo |
|---|---|---|---|
| Backend ↔ Base de datos | SQL / TCP | Interna | Síncrona |
| Backend ↔ Stripe/PayPal | REST HTTPS | TLS 1.2+ | Síncrona (checkout) + Asíncrona (webhook) |
| Backend ↔ SMTP | SMTP / API | TLS 1.2+ | Asíncrona (cola de envío) |
| Pasarela → Backend | Webhook HTTP | HMAC firma | Asíncrona (evento externo) |

## 6. Atributos de Calidad (Requisitos No Funcionales)

Los atributos de calidad definen el nivel de excelencia operativa que debe alcanzar FitPaisa. Cada atributo se vincula con los requisitos no funcionales de la ERS y se concreta en estrategias de diseño y métricas verificables.

### 6.1 Rendimiento

| Métrica | Valor objetivo | Estrategia de diseño |
|---|---|---|
| Carga del panel principal | < 2 segundos | Separación de capas · Consultas SQL optimizadas con índices · Paginación de resultados |
| Tiempo de respuesta API | < 500 ms en el percentil 95 | Diseño eficiente de endpoints · Sin operaciones bloqueantes en el hilo principal |
| Concurrencia | Múltiples usuarios activos simultáneos sin degradación apreciable | Arquitectura stateless con JWT · Conexiones a BD gestionadas por pool |
| Cálculo de macros | Respuesta inmediata (< 200 ms) | Algoritmo de Mifflin-St Jeor ejecutado en servidor sin llamadas externas |

### 6.2 Seguridad

| Área | Especificación | Implementación |
|---|---|---|
| Datos en tránsito | TLS 1.2 o superior en todas las comunicaciones | Certificado SSL/TLS en el servidor web. Redirección automática HTTP → HTTPS. |
| Autenticación | Acceso mediante correo electrónico y contraseña | Hash bcrypt con factor de coste ≥ 12. Token JWT con expiración de 2 horas. |

| Área | Especificación | Implementación |
|---|---|---|
| Autorización | Control de acceso basado en roles: USER, COACH, ADMIN | Middleware de validación de token y rol en cada endpoint protegido del backend. |
| Protección de contraseñas | Nunca almacenadas en texto plano | bcrypt en cada registro y cambio de contraseña. Nunca en logs ni respuestas API. |
| Datos de pago | Nunca almacenados en servidores propios | Delegación íntegra a Stripe/PayPal (PCI-DSS). Solo se guarda el ID externo de la transacción. |
| Validación de entradas | Sanitización y validación server-side en todos los endpoints | Validación de tipos, rangos y formatos antes de cualquier operación sobre la BD. |
| Webhooks de pago | Verificación de autenticidad del evento entrante | Validación de firma HMAC con clave secreta del proveedor antes de procesar el evento. |

### 6.3 Disponibilidad

| Métrica | Valor objetivo | Estrategia |
|---|---|---|
| Uptime mensual | ≥ 99,5% | Arquitectura web estándar preparada para despliegue en infraestructura de alta disponibilidad. |
| Recuperación ante fallos | Sin pérdida de datos | Transacciones ACID en operaciones críticas · Backups periódicos de la base de datos. |
| Gestión de errores | Degradación controlada | Manejo centralizado de excepciones · Respuestas de error estructuradas con código HTTP apropiado. |

### 6.4 Mantenibilidad

| Atributo | Estrategia |
|---|---|
| Separación de responsabilidades | Arquitectura MVC estricta. Cada módulo tiene una responsabilidad única y se comunica a través de interfaces definidas. |
| Bajo acoplamiento | Los módulos se comunican exclusivamente a través de la API REST y llamadas de servicio internas. Sin dependencias directas entre implementaciones. |

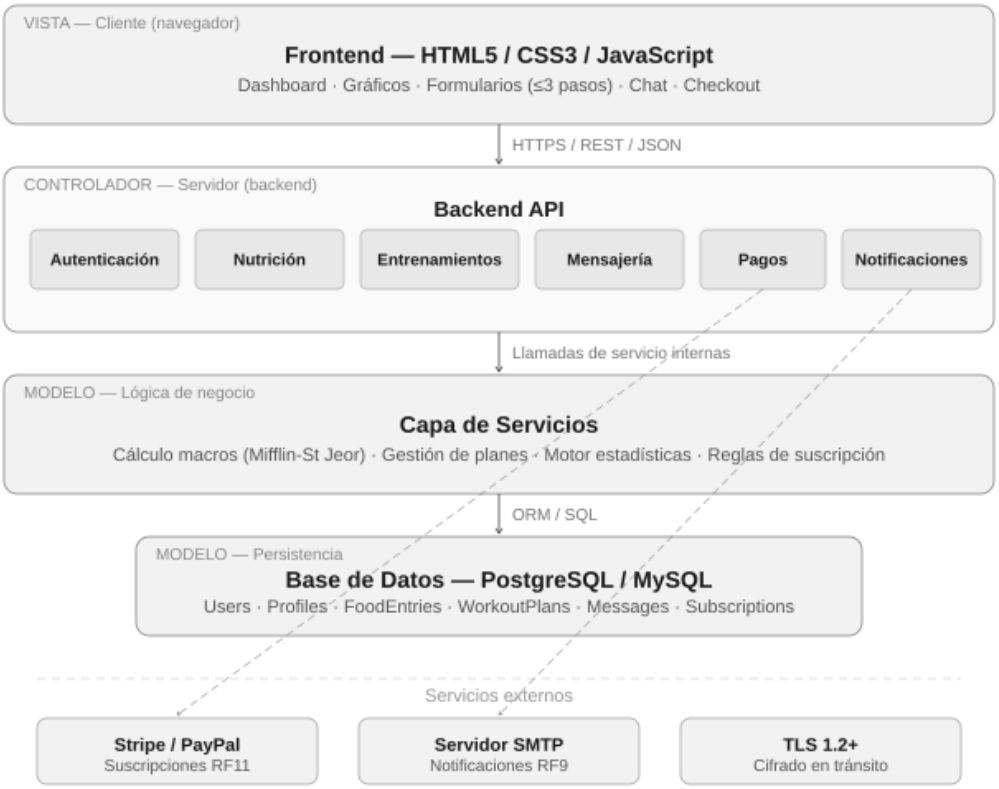
| Atributo | Estrategia |
|---|---|
| Documentación | Desarrollo basado en el estándar IEEE 830 para facilitar futuras actualizaciones o mantenimiento por otros desarrolladores. |
| Extensibilidad | Arquitectura modular que permite la ampliación de funcionalidades dentro del entorno web sin reestructuración mayor del sistema. |

### 6.5 Usabilidad y privacidad

| Atributo | Especificación | Referencia ERS |
|---|---|---|
| Flujo de acceso | El proceso de registro y primer acceso no superará los 3 pasos. | RF12 |
| Diseño responsive | La interfaz será funcional en cualquier tamaño de pantalla (escritorio, tablet, móvil). | RNF — Usabilidad |
| Privacidad de datos | Cumplimiento de normativas básicas de protección de datos personales y sensibles de salud. | RNF — Privacidad |
| Sin diagnóstico | El sistema no sustituye asesoramiento médico ni realiza diagnósticos clínicos. | ERS — Ámbito del sistema |

## 7. Diagrama de Arquitectura

El siguiente diagrama muestra la arquitectura híbrida Cliente-Servidor + MVC de FitPaisa, con los tres bloques principales (Frontend/Vista, Backend/Controlador+Modelo, Persistencia) y su relación con los servicios externos.


VISTA — Cliente (navegador)
Frontend — HTML5 / CSS3 / JavaScript
Dashboard · Gráficos · Formularios (≤3 pasos) · Chat · Checkout
HTTPS / REST / JSON
CONTROLADOR — Servidor (backend)
Backend API
Autenticación
Nutrición
Entrenamientos
Mensajería
Pagos
Notificaciones
Llamadas de servicio internas
MODELO — Lógica de negocio
Capa de Servicios
Cálculo macros (Mifflin-St Jeor) · Gestión de planes · Motor estadísticas · Reglas de suscripción
ORM / SQL
MODELO — Persistencia
Base de Datos — PostgreSQL / MySQL
Users · Profiles · FoodEntries · WorkoutPlans · Messages · Subscriptions
Servicios externos
Stripe / PayPal
Suscripciones RF11
Servidor SMTP
Notificaciones RF9
TLS 1.2+
Cifrado en tránsito